

Technical Methods Report Released-model reproduction of Nesso-1, Boltz-2, and FlashBind on the FEP+ four-kinase benchmark

Yusheng Cai

Week of August 17, 2026

Reproducible workflow: [weeks/2026-W34/code](#)

Purpose and scope

This document records the exact computational procedure I used to reproduce the released Nesso-1, Boltz-2, and FlashBind affinity results on the public FEP+ four-kinase benchmark. It is a methods companion to the explanatory Week 34 report. The emphasis here is operational reproducibility: input identity, model and checkpoint versions, inference settings, native outputs, validation gates, metric definitions, hardware, and interpretation limits.

The three calculations do not have identical levels of independence:

- **Nesso-1** was run from reconstructed protein sequences and stereochemical ligand SMILES. Protein ESM-2 embeddings were generated once per unique chain and reused across ligands.
- **Boltz-2** was run from the same reconstructed sequences and SMILES, with ColabFold/MMseqs2 MSAs generated once per unique protein chain. Boltz-2 generated both protein and ligand coordinates.
- **FlashBind** was run from the authors’ released FEP+4 archive. That archive already contained experimental PDB protein coordinates, FABind+-predicted ligand positions, ESM3 protein representations, and TorchDrug ligand representations. I reran the two released affinity scorers, but I did not rerun FABind+ docking or feature generation.

Consequently, the affinity scores can be compared on the same 87 compound identities, but the workflows should not be described as equally independent end-to-end reproductions.

1 Benchmark and frozen input policy

I used the historical OpenFF Protein–Ligand Benchmark revision `da7c3372256446222e424368be38ef3d2b55a67b`, which contains the complete 87 neutral compounds used in the model papers [1]. The current project manifest is `data/manifests/feplusplus4_87.json`, with SHA256 `5f0fb23bb97ea84f3da153e050d5913b97da9bba091fb84cd556b9894fdac1e1`.

Target series	N	Protein construct	Experimental endpoint
CDK2	16	CDK2, 303 aa; cyclin A2, 258 aa	IC50
TYK2	16	TYK2, 302 aa	Ki
JNK1	21	JNK1, 370 aa; JIP1 peptide, 11 aa	IC50
p38 α	34	p38 kinase, 372 aa	IC50

Table 1: Frozen affinity benchmark. Each record pairs the indicated target construct with one ligand.

Protein sequences were taken from the deposited benchmark complexes. I kept the cyclin A2 partner for CDK2 and the JIP1 peptide for JNK1. Ligands were represented by canonical, stereochemically explicit SMILES and restricted to formal charge zero. Experimental values were converted to μM and stored as

$$y_i = \log_{10} \left(\frac{C_i}{\mu\text{M}} \right).$$

Smaller values correspond to stronger measured potency. The IC50 and Ki labels were retained as different assay types; neither was relabeled as an equilibrium binding free energy.

The experimental labels were not supplied to Nesso-1 or Boltz-2 inference. The FlashBind archive included a label file, but I did not use it to generate predictions or as the source of the primary analysis labels. I joined all predictions to the frozen project manifest only after output collection.

2 Common execution and audit procedure

All three models were executed on one NVIDIA GeForce RTX 3080 with 10,240 MiB VRAM and NVIDIA driver 535.230.02. Each model used an isolated environment; external `PYTHONPATH` settings were unset except when FlashBind’s source directory was deliberately supplied. Every substantive command was wrapped by `scripts/run_with_provenance.py`, which recorded the resolved command, Git revision, software environment, timestamps, wall time, GPU observations, return code, and failure status in `run.json`.

The same staged rule was applied to each workflow:

1. Verify the pinned source, checkpoint, input, and environment files.
2. Run a small smoke or target-level pilot and require finite native outputs.
3. Expand to the frozen 87 records without changing scientific settings.
4. Preserve native model output files before parsing or normalization.
5. Mark missing or failed records explicitly; do not silently omit them.
6. Join experimental labels by immutable sample ID only during analysis.

The top-level inference entry point was:

```
bash weeks/2026-W34/code/04_run_inference.sh --model=all --execute
```

The model-specific commands described below are defined in that script and are printed without execution when `--check` is used.

3 Nesso-1 procedure

3.1 Pinned implementation and input construction

I used Nesso-1 version 1.0.0 at source revision `f0156e9a22326448684bae09ee96f73415902dcd`. The released checkpoint was resolved to Hugging Face commit `1896c84c7186c506c7efd79051480809d51098bf`; its model SHA256 was `9928a8a824d147d665e76656804af1cd91c86731516d0b561f9fd1c91ee45622`. Protein sequence embeddings used `facebook/esm2_t33_650M_UR50D` at revision `08e4846`.

One YAML input was generated per protein–ligand record. Each contained the exact protein chain sequence or sequences, one ligand as stereochemical SMILES, and an affinity request naming that ligand as the binder. No MSA, experimental protein coordinates, known pocket, template, or experimental label was provided. ESM-2 embeddings were generated once for each unique protein chain and reused across every ligand in the same target series.

3.2 Inference settings and model flow

The released model was run with seed 42, mixed bfloat16 precision, five recycling steps, and the default two-stage pocket refinement. The first trunk pass processed the complete tokenized protein–ligand system and produced protein–ligand distance distributions. Protein tokens with predicted ligand distance up to 22 Å were considered for a refinement crop containing at most 256 total protein-residue and ligand-heavy-atom tokens. Five subsequent recycled trunk passes refined this cropped pair representation. A tighter 15 Å predicted-distance region was then supplied to the two released affinity heads [2].

The resolved settings were:

Setting	Value
Accelerator / devices	GPU / 1
Precision	<code>bf16-mixed</code>
Seed	42
Recycling	5 feedback steps after the initial pass
Pocket refinement	enabled; 22 Å predicted-distance cutoff
Refinement token budget	256 total tokens
Final affinity protein cutoff	15 Å
Optional accelerated kernels	disabled with <code>--no_kernels</code>

Table 2: Nesso-1 settings used for all four target series.

For each record, I preserved the ensemble mean `affinity_pred_value`, the component outputs `affinity_pred_value1` and `affinity_pred_value2`, the binary binder probability, and interface entropy diagnostics. The continuous ensemble mean was used for affinity ranking; binder probability was analyzed as a separate output and was never substituted for the continuous value.

All 87 records completed. The four target runs required a summed 881.31 s of wall time; the largest observed total GPU-memory use was 9,292 MiB. These are local execution measurements, not architecture-independent speed benchmarks.

4 Boltz-2 procedure

4.1 Pinned implementation, inputs, and MSAs

I used Boltz 2.2.1 source revision `b1ebfc46ecf57f5414e0d1a6f9027bbb122c53bc`. The structure checkpoint `boltz2_conf.ckpt` had SHA256 `090e82ac8c92f5e943fa1b39e7410a44027bea7243c0bbb3caa67a77fc1428e1`; the affinity checkpoint `boltz2_aff.ckpt` had SHA256 `dcc5cd3722b1c9eaa34267e4ae32f55cbbf1963f4c19319381ccfa30fdd2ca9e`.

The sequence and SMILES identities were the same as for Nesso-1. In addition, I submitted each unique protein chain sequence to the public ColabFold MMseqs2 server and reused the resulting MSA for every ligand in that target. No structural template, pocket constraint, experimental coordinate, or experimental affinity label was supplied [3].

The parsed MSA pool was capped at 8,192 rows. Passing the complete retained MSA to the trunk exceeded the available 10 GiB GPU memory, so the reported `boltz2_msa1024` protocol enabled Boltz’s released stochastic MSA subsampling option and selected 1,024 rows from that pool for each model pass. The original full-MSA failure was preserved as a hardware-feasibility result; it was not counted as an accuracy result.

4.2 Structure and affinity sampling

Boltz-2 jointly generated protein and ligand heavy-atom coordinates. For each record, I used three structure recycles, one structure diffusion sample with 200 sampling steps, step scale 1.5, and one sample in memory at a time. The affinity calculation used five diffusion samples with 200 sampling steps. I used seed 42, disabled optional accelerated kernels, did not enable potentials, and did not apply the optional molecular-weight affinity correction.

Setting	Value
Structure recycles	3
Structure samples / steps	1 sample / 200 steps
Affinity samples / steps	5 samples / 200 steps
Step scale	1.5
MSA pool / trunk sample	at most 8,192 parsed / 1,024 sampled rows
Templates / constraints / potentials	none / none / disabled
Affinity molecular-weight correction	disabled
Output coordinates	mmCIF
Seed	42

Table 3: Boltz-2 MSA-1024 settings used for all 87 records.

I retained each sampled complex, confidence outputs, continuous `affinity_pred_value`, and `affinity_probability_binary`. The continuous field was used for affinity ranking; the binder probability remained separate. All 87 MSA-1024 records completed. The four target runs required a summed 5,921.84 s, with a maximum observed total GPU-memory use of 9,988 MiB.

5 FlashBind released-pose scoring procedure

5.1 Precisely what was and was not rerun

I used FlashBind source revision `f161268176237ab6ce5757031a8c1b93937e0d37`. The two released continuous value checkpoints came from Hugging Face revision `fa6362c85b3350109af4634e3e2c364644338b3c`; their SHA256 values were `ca87f84dedd4642da693a508b1119c802caa370e356dad31c47b8a708821e553` and `59fa2686b0859a632862aec21ea69f2bebb1ab104e48606fe739c929368d488d`.

The released FEP+4 archive was pinned at revision `50b1511e080236d80f9b8ff1e4d0cd38bb2480b9`. For each record it already provided the inputs needed by the final scorer:

- one experimental PDB protein structure shared within each target;
- one FABind+-predicted ligand position and orientation in SDF format;
- a precomputed ESM3 representation for the protein; and
- precomputed TorchDrug atom features for the ligand.

I therefore reran only the FlashBind scoring stage. I did not search the PDB, prepare the protein, rerun FABind+, regenerate ligand coordinates, rerun ESM3, or regenerate TorchDrug features. This distinction is essential when comparing the result with Boltz-2, which generated the complete complex in this study [4].

Before inference, I matched all 87 archive molecules to the frozen manifest by target and InChIKey. All 87 chemical identities matched. The archived labels matched the project labels with maximum absolute difference 4.44×10^{-16} , and the four archived PDB files were byte-identical to the local benchmark copies. These were identity audits; the archived labels were not model inputs.

5.2 Scoring and calibration

FlashBind was run in continuous `value` mode. The scorer used the supplied three-dimensional ligand position to construct a local protein region with the released 20 Å distance-threshold setting. Both value checkpoints were evaluated and their native outputs were averaged. A one-record CDK2 smoke test first required finite `pred_value`, `pred_value_raw`, and molecular weight; the complete 87-record run began only after this passed.

The raw neural-network output was retained as `pred_value_raw`. The released implementation applies the molecular-weight calibration

$$\text{pred_value} = 2.177929 \text{pred_value_raw} - 0.218761 \text{MW}^{0.3} + 1.472271.$$

The calibrated `pred_value`, documented as approximately $\log_{10}(\text{IC}_{50}/\text{K}_i/\text{K}_d \text{ in } \mu\text{M})$, was used for the primary ranking analysis. Lower values indicate stronger predicted potency. This mixed-assay score is not a thermodynamic ΔG_{bind} .

The upstream environment specified PyTorch 2.7.1 with CUDA 12.6, which was incompatible with the local NVIDIA driver. I used an isolated PyTorch 2.5.1 / CUDA 12.1 environment while retaining the authors’ code, weights, inputs, and model settings. The calculation is therefore a numerical reproduction of the released scorer, not a bitwise environment reproduction.

All 87 records completed in 24.90 s, with observed total GPU-memory use of 3,576 MiB. That runtime excludes the upstream FABind+ docking, ESM3 embedding, and TorchDrug featurization that had already been performed by the authors.

6 Output normalization and affinity metrics

Native output JSON files were preserved beneath `runs/`. I converted them to a common CSV schema with `scripts/collect_affinity_outputs.py` for Nesso-1 and Boltz-2 and `scripts/collect_flashbind_outputs.py` for FlashBind. Parsing checked that the designated continuous output and binder probability were finite and kept the path to the original native file. No per-target rescaling or post-hoc sign reversal was applied.

For each target t , I calculated

$$r_t = \text{Pearson}(y_{t,i}, \hat{y}_{t,i}), \quad \bar{r} = \frac{\sum_t n_t r_t}{\sum_t n_t}.$$

Thus, the primary result is the compound-count-weighted mean of four within-target Pearson correlations. It does not correlate compounds measured in unrelated assays across targets. Spearman ρ and Kendall τ were calculated as complementary rank statistics. I estimated uncertainty using 2,000 bootstrap replicates with seed 20260817, resampling compounds with replacement independently inside each target.

Model/protocol	Pearson r	Spearman ρ	Kendall τ
Nesso-1	0.807	0.819	0.641
Boltz-2 MSA-1024	0.627	0.666	0.497
FlashBind, released FABind+ poses	0.533	0.515	0.359

Table 4: Compound-count-weighted within-target affinity statistics over all 87 records. The corresponding paper Pearson values were 0.80, 0.66, and 0.53.

7 Structure-recovery analysis

The structure analysis used the 16 records for which the exact protein–ligand identity had a deposited cocrystal. Nesso-1 was excluded because it does not produce a final Cartesian complex. Boltz-2 generated both protein and ligand coordinates. FlashBind used an experimental PDB protein structure and a ligand position previously predicted by FABind+.

For each pair, I matched protein chains by sequence. The experimental pocket was defined as residues having any heavy atom within 5 Å of the crystallographic ligand. I calculated three components:

1. global protein C α RMSD after fitting all matched C α atoms;
2. pocket protein C α RMSD after fitting matched pocket C α atoms; and
3. symmetry-corrected ligand heavy-atom RMSD after applying the protein-pocket transform, without independently fitting the ligand.

For the ligand calculation, I minimized over complete bond-order- and chirality-compatible graph mappings π ,

$$\text{RMSD}_{\text{pose}} = \min_{\pi} \sqrt{\frac{1}{N} \sum_i \left\| R\mathbf{x}_i + \mathbf{t} - \mathbf{x}_{\pi(i)}^{\text{ref}} \right\|^2},$$

where R and $\mathbf{t}$ were determined only from the protein pocket fit. This prevents the experimental ligand coordinates from improving the alignment being evaluated.

Median RMSD over 16 pairs (Å)	Boltz-2	FlashBind/FABind+
Global protein C α , fitted	1.231	0.272
Pocket protein C α , fitted	0.400	0.208
Ligand heavy atoms after pocket fit	0.898	0.634

Table 5: Protein and ligand coordinate comparison. FlashBind’s protein values compare experimental PDB inputs with ligand-specific cocrystals; they are not FlashBind protein-prediction errors.

8 Reproducible command sequence

From the repository root, the complete preserved workflow is:

```
bash weeks/2026-W34/code/01_setup_models.sh --check
bash weeks/2026-W34/code/02_download_assets.sh --check
bash weeks/2026-W34/code/03_prepare_inputs.sh --check
bash weeks/2026-W34/code/04_run_inference.sh --model=all --check

# Expensive execution, only when assets and GPU are available:
bash weeks/2026-W34/code/04_run_inference.sh --model=all --execute

bash weeks/2026-W34/code/05_collect_outputs.sh --execute
bash weeks/2026-W34/code/06_analyze_affinity.sh --execute
bash weeks/2026-W34/code/07_analyze_poses.sh \
  --execute --allow-rcsb-download
```

MSA generation on a new machine additionally requires explicit permission to submit protein sequences to the public ColabFold service:

```
bash weeks/2026-W34/code/03_prepare_inputs.sh \
  --execute --allow-msa-server
```

The scripts preserve completed runs, default to a check or command-preview mode, and keep checkpoints, MSAs, caches, raw datasets, and raw prediction directories outside Git. The repository tracks the frozen manifests, configuration files, adapters, metric implementations, compact tables, figures, report sources, and tests.

9 Interpretation limits

This work establishes that the released checkpoints and preserved inputs can produce the reported within-target ranking behavior under the documented protocols. It does not establish universal affinity accuracy, prospective generalization, or prediction of a standard binding free energy. The benchmark contains four public congeneric kinase series and may overlap model training chemistry. IC50 and Ki are assay-dependent quantities. The Boltz-2 result uses a documented 1,024-row MSA trunk sample, and the FlashBind result uses the authors’ already docked FABind+ poses and feature archive.

The technical conclusion is therefore limited but reproducible: Nesso-1 and Boltz-2 were rerun from reconstructed sequence/SMILES inputs, FlashBind’s final scoring stage was rerun from authors’ released prepared complexes, and all three produced complete 87-record continuous affinity outputs that closely reproduced their respective paper-level Pearson summaries.

References

- [1] David F. Hahn et al. Best practices for constructing, preparing, and evaluating protein-ligand binding affinity benchmarks. *Living Journal of Computational Molecular Science*, 4(1):1497, 2022. doi: 10.33011/livecoms.4.1.1497. URL <https://doi.org/10.33011/livecoms.4.1.1497>.
- [2] Nikhil Shenoy, David Errington, Emmanuel Bengio, Kacper Kapuśniak, Kerstin Klaeser, Yui Tik Pang, Vladimir Radenkovic, Prudencio Tossou, Therence Bois, Andrew Wedlake, and Francesco Di Giovanni. Nesso-1: Accelerating open-source binding affinity predictions. *bioRxiv*, 2026. doi: 10.64898/2026.08.01.742196. URL <https://doi.org/10.64898/2026.08.01.742196>. Preprint, version 1, posted 3 August 2026.
- [3] Saro Passaro, Gabriele Corso, Jeremy Wohlwend, Mateo Reveiz, Stephan Thaler, Vignesh Ram Somnath, Noah Getz, Tally Portnoi, Julien Roy, Hannes Stark, David Kwabi-Addo, Dominique Beaini, Tommi Jaakkola, and Regina Barzilay. Boltz-2: Towards accurate and efficient binding affinity prediction. *bioRxiv*, 2025. doi: 10.1101/2025.06.14.659707. URL <https://doi.org/10.1101/2025.06.14.659707>. Preprint, version 1, posted 18 June 2025.
- [4] Songlin Jiang, Yifan Chen, Aarti Krishnan, Yu Zhang, and Wengong Jin. Flashbind: Towards accurate and efficient structure-based virtual screening. *bioRxiv*, 2026. doi: 10.64898/2025.12.22.695983. URL <https://doi.org/10.64898/2025.12.22.695983>. Preprint, version 2, posted 8 April 2026; version 1 was titled FlashAffinity.